

AI Yoga Posture & Wellness Companion

System Architecture — base implementation (Vrikshasana) · Vishnu Patil

1 Runtime pipeline — the life of one frame

Everything below runs 28 times a second on one CPU core. Timings are measured medians over 50 frames at 1000×667 (Intel i7-13xxx, no GPU).

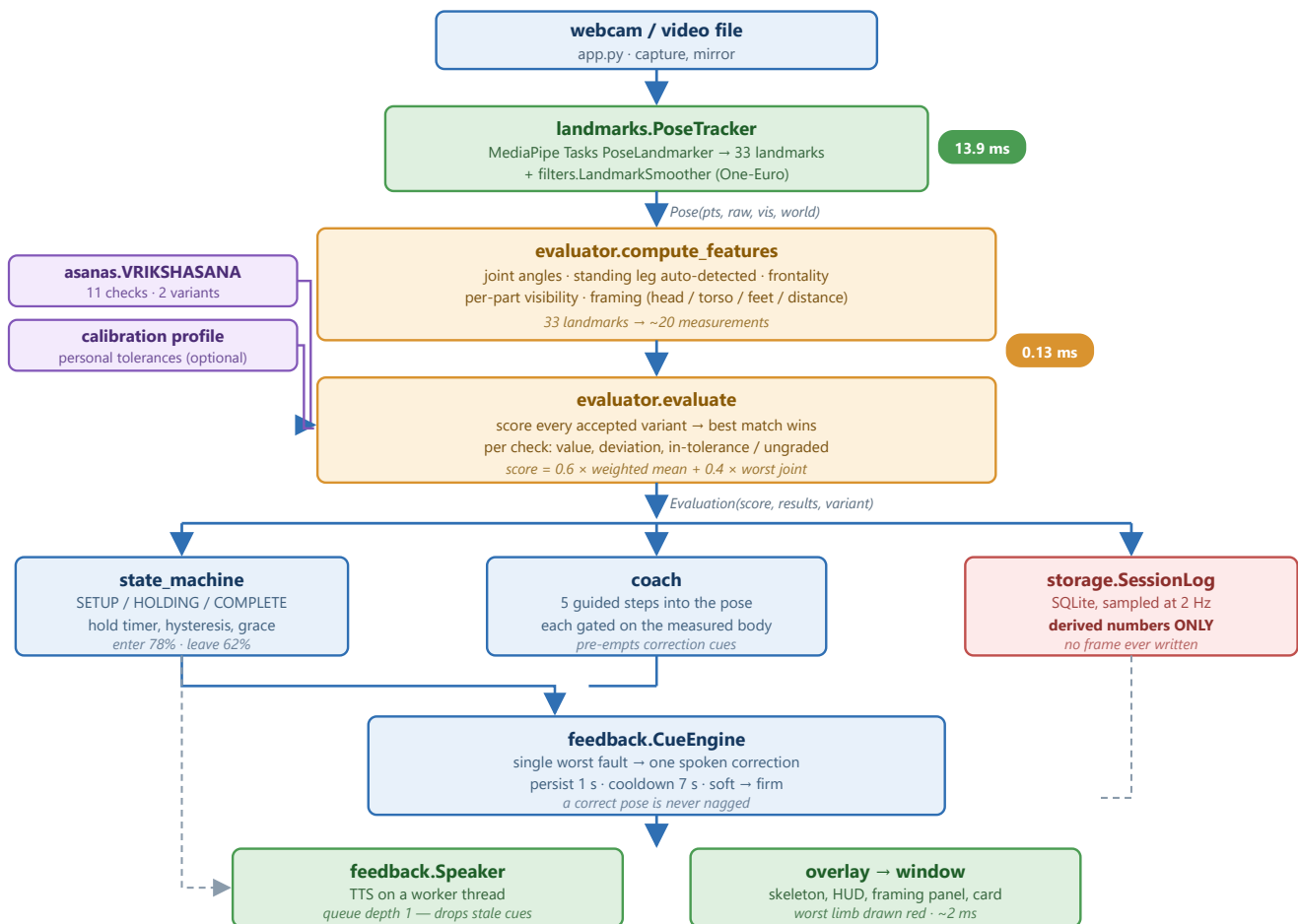


Figure 1 — Per-frame runtime pipeline. Solid arrows carry data; dashed arrows carry state.

Where the time goes. The neural network costs 13.9 ms; the whole scoring layer this project contributes costs **0.13 ms** — roughly 1% of the budget. The pipeline is bounded entirely by MediaPipe, which is what makes a 30 fps camera comfortable on a laptop with no GPU.

2 Module structure

Thirteen modules, ~4,200 lines. The dependency graph is deliberately shallow: no module imports more than three siblings, and the asana definitions import nothing at all.

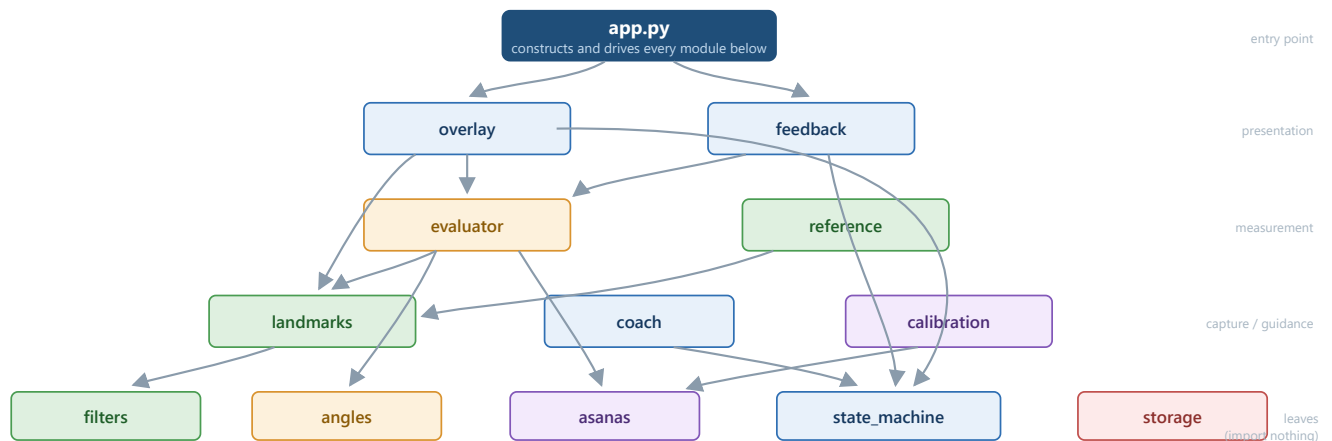


Figure 2 — Module dependency graph, laid out in layers. Every arrow is a real import extracted from the source; a module sits above everything it depends on.

Module	Lines	Responsibility
<code>landmarks.py</code>	143	Wraps the MediaPipe Tasks API. The rest of the code never sees a mediapipe type.
<code>filters.py</code>	89	One-Euro filter: heavy smoothing when still, almost none when moving fast.
<code>angles.py</code>	62	Joint geometry in image space, written so asanas read like a teacher's cues.
<code>asanas.py</code>	262	Pure declaration. Checks, targets, tolerances, variants. No logic, no imports.
<code>evaluator.py</code>	336	Measure the body; grade it joint by joint; pick the best-fitting variant.
<code>state_machine.py</code>	155	Hold timer with hysteresis and a grace window.
<code>coach.py</code>	246	The guided spoken entry into the pose, gated on measured body position.
<code>feedback.py</code>	180	Which cue, when to say it, and the text-to-speech worker thread.
<code>overlay.py</code>	332	Skeleton, HUD, framing panel, reference card.
<code>reference.py</code>	150	The ideal figure, shared by the on-screen card and the self-test.
<code>calibration.py</code>	92	Personal tolerances learned from the practitioner's own best attempt.
<code>storage.py</code>	122	SQLite practice log. Derived numbers only.

Why the graph matters. `asanas.py` holds no logic, so adding a second asana is one declaration and one fitting run — the engine does not change. And because `reference.py` feeds both the on-screen card and the test suite, the figure the practitioner is shown is *by construction* the figure the scorer accepts.

3 Pose state machine

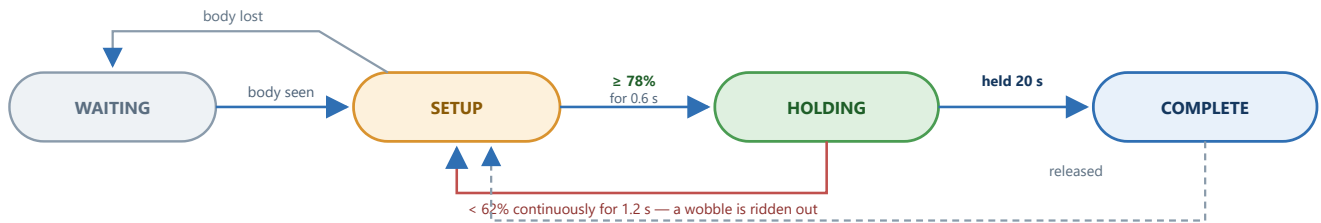


Figure 3 — Asymmetric thresholds: 78% to enter, 62% to leave, plus a 1.2 s grace window.

4 Offline pipeline — where the reference angles come from

This runs once, not per frame. It is what makes the numbers in `asanas.py` defensible rather than guessed.

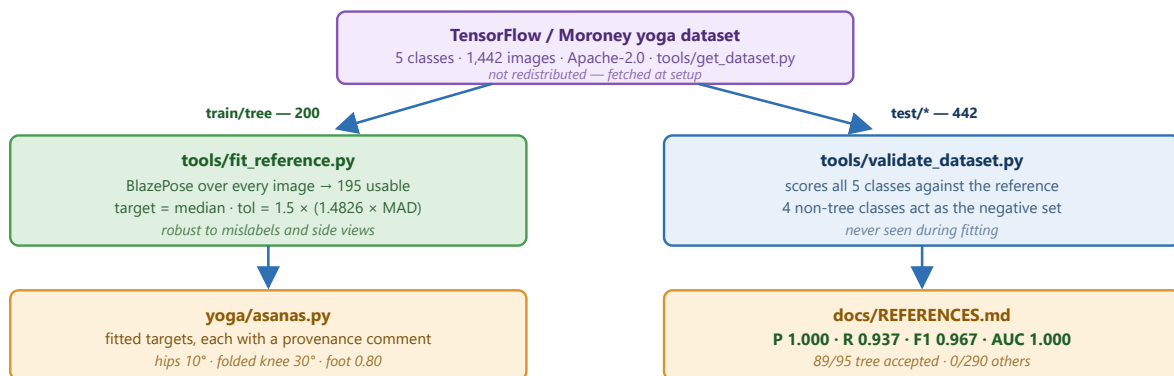


Figure 4 — The reference is fitted on the train split only; the test split is never seen during fitting.

Why fit at all? No standard table of ideal joint angles per asana exists in the literature — the papers that do angle-based correction each derive their own from data and mostly never publish the numbers. Three fitted values contradicted the obvious guess: the hips sit **10° off level**, not 0°; the folded knee closes to **30°**, not 45°; the foot sits at **0.80** of the standing leg, not 0.70.

5 Verification and delivery

Path	What it proves
<code>tools/selftest.py</code>	66 assertions on synthetic skeletons of known geometry — no camera, no dataset. Angles recover to within 0.6°; each wrong pose names the right fault; the hold timer rides out a 0.7 s wobble but ends on a 2 s loss.
<code>tools/simulate_session.py</code>	A scripted 90-second practice driven through the <i>real</i> evaluator, state machine, cue engine and logger. Asserts two holds complete.
<code>tools/analyse.py</code>	Any clip or photo → per-frame CSV of every joint. The harness for instructor-labelled ground truth (Stage 5).
<code>bootstrap.py</code>	Self-healing setup: every requirement is a <i>check</i> paired with a <i>repair</i> , looped until green. Runs the self-test as its final gate.
<code>Dockerfile</code>	Runs the self-test during the image build — a green image is a green engine.

Privacy, as built. No frame is ever written to disk; frames are decoded, measured and discarded. The practice log holds joint angles, deviations, scores, hold durations and cue text — nothing else — in a local SQLite file. Every stage runs on the machine; there is no network call at run time.

Scope. This is the asana-trainer track, complete, for one asana. The desk-posture and contactless-vitals tracks in the methodology are not built yet. The validation above proves the system identifies Vrikshasana against other asanas; it does not yet prove it grades a *good* Vrikshasana against a poor one — that needs instructor-labelled footage, for which the harness already exists.